Practical 4: Modeling UML Class Diagrams

Objectives

- Shows static structure of classifiers in a system
- Diagram provides a basic notation for other structure diagrams prescribed by UML
- Helpful for developers and other team members too
- Business Analysts can use class diagrams to model systems from a business perspective

A UML class diagram is made up of:

- A set of classes and
- A set of relationships between classes

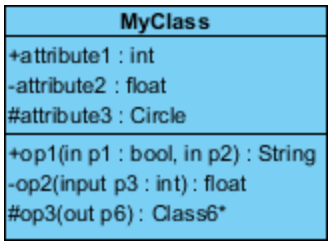
Class Notation

A class notation consists of three parts:

1. **Class Name**
   - The name of the class appears in the first partition.
2. **Class Attributes**
   - Attributes are shown in the second partition.
   - The attribute type is shown after the colon.
   - Attributes map onto member variables (data members) in code.
3. **Class Operations** (Methods)
   - Operations are shown in the third partition. They are services the class provides.
   - The return type of a method is shown after the colon at the end of the method signature.
   - The return type of method parameters is shown after the colon following the parameter name.
   - Operations map onto class methods in code

Visibility of Class attributes and Operations

- + denotes public attributes or operations
- - denotes private attributes or operations
- # denotes protected attributes or operations
- ~ denotes package attributes or operations

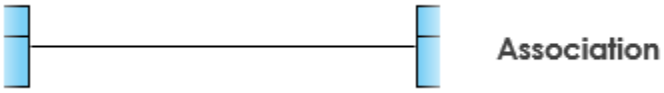


Relationships

Existing relationships in a system describe legitimate connections between the classes in that system.

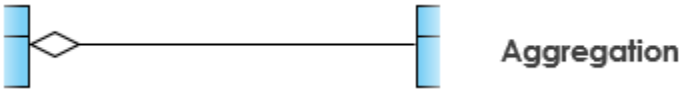
- **Association**

- A class that sits between two other classes to hold attributes specific to the association itself. For instance, rather than connecting Member directly to Book, a Loan class acts as an association class holding checkoutDate and dueDate.



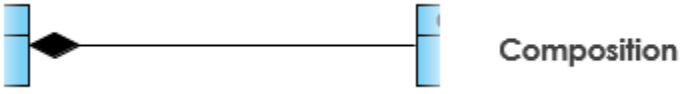
- **Aggregation**

- A "has-a" relationship (represented by an empty diamond). For example, a Library contains a collection of Book items, but the books can still exist if the library is destroyed.



- **Composition**

- Shows total ownership and lifecycle dependency. The child cannot live without the parent. For Example a single Library system is composed of multiple LibraryBranch objects. If the parent Library shuts down, its physical branches cease to exist in the system.



- **Multiplicity**

- How many objects of each class take part in the relationships and multiplicity can be expressed as:

- Exactly one - 1
- Zero or one - 0..1
- Many - 0..* or *
- One or more - 1..*
- Exact Number - e.g. 3..4 or 6
- Or a complex relationship - e.g. 0..1, 3..4, 6.* would mean any number of objects other than 2 or 5

- **Example: User and Book:** 1 to 0..*. One library user can borrow zero or many books, but a specific physical book is borrowed by exactly one user at a time.

Case Study: A Library Information System for SE VLabs Institute: